

동시성을 위한 Go

3편: Go 동시성 구성 요인

남궁명수

[03-1] 고루틴	4
(1) 고루틴은 무엇인가?	4
(2) 고루틴의 숨겨진 작동 원리	5
(3) 포크-조인 모델과 실행 보장 문제	6
(4) 고루틴과 클로저가 만날 때의 함정	7
(5) 고루틴의 무게 측정과 스케일	9
(6) 눈부시게 빠른 문맥 전환 성능	11
(7) 결론	12
[03-2] sync 패키지	13
(1) 개요	13
(2) 대기 조율사(WaitGroup)	13
(3) 상호배제 자물쇠(Mutex and RWMutex)	16
(4) 조건 대기방(Cond)	21
(5) 단 한번의 실행 보장(Once)	26
(6) 객체 재활용 창고(Pool)	29
[03-3] 채널	36
(1) 채널의 기본 개념과 생성	36
(2) 단방향 채널의 규칙과 타입 안정성	37
(3) 채널의 동기화 메커니즘과 데드락	39
(4) 채널 닫기와 복수 반환값 활용	41
(5) 채널 순회 패턴과 다중 고루틴 동시 깨우기	43
(6) 버퍼를 가진 채널의 작동 원리	45
(7) Nil 채널의 패닉과 동작 상태 총정리	48
(8) 채널 소유권 설계 원칙	49
[03-4] select	52
(1) select 문의 기본 구조	52
(2) 복수 채널이 동시에 준비될 때의 무작위 법칙	54

(3) 타임아웃 구현 패턴	56
(4) 비블로킹 처리를 위한 default 구문	57
(5) 영원히 멈추는 특수 케이스	59
(6) 결론	60
[03-5] GOMAXPROCS	61
(1) GOMAXPROCS 이름에 숨겨진 오해	61
(2) 이 값을 개발자가 직접 조절해야 하는 이유	62
(3) GOMAXPROCS 레버 조절 시 주의할 점	63
(4) 결론	64

[03-1] 고루틴

(1) 고루틴은 무엇인가?

고루틴은 Go 프로그램에서 동시성을 관리하는 가장 기본적이고 핵심적인 단위이다.

사실 모든 Go 프로그램은 자신도 모르게 이미 최소한 하나 이상의 고루틴을 쓰고 있다.

바로 프로그램이 켜지자마자 자동으로 시작되는 ‘메인 고루틴’이다.

여러분도 문제를 풀다 보면 조만간 이 고루틴의 강력한 도움을 받게 될 것이다.

그렇다면 고루틴의 정체는 무엇일까?

아주 쉽게말해, 고루틴은 다른 코드들과 ‘동시에’ 실행되는 하나의 함수일 뿐이다.

기억하나? 물리적으로 완전히 똑같은 순간에 도는 ‘병렬’이 아닐 수도 있다.

어떤 함수를 고루틴으로 실행하는 방법은 세상에서 가장 쉽다.

함수 호출문 앞에 단 한 글자, go 키워드만 붙여주면 끝이다.

```
func main() {  
    go sayHello() // 고루틴으로 독립적인 일꾼을 띄웁니다!  
    // 메인 고루틴은 아래로 내려가 자기 할 일을 계속합니다.  
}  
  
func sayHello() {  
    fmt.Println("hello")  
}
```

이름이 없는 함수인 익명 함수도 고루틴으로 만들 수 있다.

아래 코드는 위와 완전히 똑같이 작동한다.

```
go func() {  
    fmt.Println("hello")  
}() // 주의: 익명 함수이므로 정의하자마자 끝에 ()를 붙여서 즉시 실행해야 합니다
```

또는 함수를 변수에 담아서 띄울 수도 있다.

```
sayHello := func() {  
    fmt.Println("hello")  
}  
  
go sayHello() // 변수에 담긴 함수를 고루틴으로 실행
```

함수 하나와 키워드 한 단어만으로 동시성 코드를 똑딱 만들었다.

믿기 힘들겠지만, 고루틴을 시작하는 방법은 이게 전부다.

물론 이 고루틴들을 어떻게 조율하고, 동기화하고, 관리할지에 대해서는 할 이야기가 많지만, 활용하는 법 자체는 이게 끝이다.

(2) 고루틴의 숨겨진 작동 원리

고루틴에 몇 가지 의문이 들 수 있다.

- 그렇다면 보이지 않는 무대 뒤편에서는 도대체 무슨 일이 벌어지고 있을까?
- 고루틴은 OS 스레드일까?
- 아니면 그린 스레드(Green Thread)일까?
- 우리가 몇 개나 만들 수 있을까?

고루틴은 Go 언어만의 아주 독특한 무기이다.

결론부터 말하자면, 고루틴은 OS 스레드도 아니고 완벽한 그린 스레드도 아니다.

이보다 한 차원 높은 추상화 단계인 코루틴(Coroutines)의 일종이다.

코루틴이란?

중간에 강제로 끼어들어 멈출 수 없고(비선점형), 자기 스스로 “나 여기서 잠시 쉴게”하고 멈추거나 “다시 시작할게”하고 들어올 수 있는 여러 진입점을 가진 동시성 서브루틴(함수)을 말한다.

고루틴이 특별한 이유는 Go의 시스템 심장부(Runtime)와 완벽하게 한 몸으로 결합되어 있기 때문이다. 고루틴은 스스로 “나 여기서 멈출래”라고 고집 부리지 않는다.

Go 런타임이 고루틴의 행동을 가만히 지켜보고 있다가,

고루틴이 네트워크나 파일 입출력(I/O) 등으로 인해서 멈춰 서는 순간을 포착해서 자동으로 고루틴을 일시정지시키고, 일이 풀리면 자동으로 다시 깨워준다.

멈추는 순간에만 개입하므로, 일종의 우아한 파트너십을 맺은 셈이다.
그래서 고루틴은 코루틴 중에서도 아주 특별한 클래스로 분류된다.

또한, Go언어는 고루틴들을 효율적으로 컴퓨터 두뇌에 배치하기 위해 M:N 스케줄러라는 시스템을 사용한다. 이는 M개의 그린 스레드(고루틴)를 N개의 실제 OS 스레드에 나누어 엮는 기술이다. 고루틴이 아무리 많아도 스케줄러가 알아서 살아있는 OS 스레드 위로 일을 골고루 분배해 주기 때문이다. 우리는 스레드가 멈추는 복잡한 하드웨어 이슈를 전혀 신경 쓰지 않아도 된다.

(3) 포크-조인 모델과 실행 보장 문제

Go언어의 동시성은 ‘포크-조인(Fork-Join) 모델’이라는 논리적 구조를 따른다.

- 포크: 길을 가다 포크처럼 갈래길이 갈라지듯, 프로그램의 어느 한 시점에서 자식 가지(고루틴)를 쪼개어 동시에 실행하는 행동을 뜻한다.(go 키워드가 포크 역할)
- 조인: 쪼개져서 각자 일하던 자식 가지들이 미래의 어느 시점에 다시 부모의 품으로 합쳐지는 것을 뜻한다. 합쳐지는 지점을 ‘조인 포인트’라고 부른다.

다시 우리가 처음에 봤던 단순한 고루틴 코드를 복습해보자.

```
sayHello := func() { fmt.Println("hello") }  
go sayHello() // 포크(Fork) 발생  
// 메인 루프 계속 실행...
```

이 코드에는 치명적인 버그가 하나 숨어있다. 바로 조인 포인트가 없다는 점이다.

자식 일꾼(sayHello)을 밖으로 던져놓기만(Fork)했을 뿐, 부모(메인 함수)가 자식을 기다려주지 않는다. 메인 함수는 0.00001초 만에 밑으로 내려가서 프로그램을 완전히 종료시켜 버릴 것이다. 결국 자식 고루틴은 채 실행되기도 전에 프로그램이 꺼져버려서, 화면에 “hello”라는 글자가 찍히는 것을 구경조차 못할 확률이 99%이다.

1장에서 배운 것처럼, 대충 슬립(time.Sleep)을 걸다가 기다리는 것은 정답이 아니다. 그건 조인 포인트가 아니라 그저 “실행되길 기도하는 경쟁 조건”을 만드는 임시방편일 뿐이다. 프로그램의 정답을 보장하고 데이터 경쟁을 없애려면 명확한 ‘조인 포인트’를 심어두어야 한다.

이 조인 포인트를 만드는 도구가 바로 뒤에서 배울 `sync.WaitGroup`이다.

```
var wg sync.WaitGroup // 조율사 등장

sayHello := func() {
    defer wg.Done() // 3. 자식: "저 일 다 끝났어요" 알림
    fmt.Println("hello")
}

wg.Add(1)    // 1. 부모: "기다려야 할 일꾼 1명 추가"
go sayHello() // 2. 포크(Fork)!
wg.Wait()    // 여기가 바로 조인 포인트(Join Point) 자식이 Done을 외칠 때까지 부모가 꿈
              // 짍 않고 기다립니다.
```

이렇게 하면 조인 포인트가 안전하게 뚫어주어야 비로소 화면에 확실하게 hello가 찍히는 올바른 프로그램이 완성된다.

(4) 고루틴과 클로저가 만날 때의 함정

고루틴을 익명 함수로 만들 때 가장 많이 쓰이면서도,
초보자들이 100% 한 번씩 코가 깨지는 클로저(Closure)의 함정을 알아보자.

클로저란

함수 주변의 변수를 자기 안으로 꼭 감싸서 가져가는 성질을 말한다.
만약 고루틴 내부에서 바깥의 변수를 그냥 가져다 쓰면, 변수의 복사본을 쓸까,
아니면 원본 주사를 공유할까?

코드로 확인해 보자.

```
var wg sync.WaitGroup
salutation := "hello"

wg.Add(1)
go func() {
    defer wg.Done()
    salutation = "welcome" // 바깥 변수 원본을 직접 수정해 버립니다
}()
```

```
wg.Wait()
fmt.Println(salutation)
```

출력 결과는 welcome이다.

즉 고루틴은 자신이 태어난 메모리 주소 공간을 공유하기 때문에 원본 변수를 그대로 건드린다.

그럼 이 성질 때문에 일어나는 Go언어의 가장 유명한 대참사 코드를 보여주겠다.

아래 프로그램은 무엇을 출력할까?

```
var wg sync.WaitGroup

for _, salutation := range []string{"hello", "greetings", "good day"} {
    wg.Add(1)
    go func() {
        defer wg.Done()
        fmt.Println(salutation) // 루프 변수를 그대로 가져다 씀!
    }()
}

wg.Wait()
```

인사말 3개가 골고루 출력될 것 같지만, 실제 결과는 충격이다.

```
good day
good day
good day
```

왜 이런일이 일어날까? 컴퓨터가 루프(for)를 도는 속도는 상상을 초월하게 빠르다.

고루틴 일꾼 3명이 채 준비운동을 마치고 실행되기도 전에 이미 루프는 마지막 세 번째 단어인 good day까지 다 돌아버린 상태이다.

고루틴 내부에서는 바깥의 salutation 변수 주소를 쳐다보고 있기 때문이다.

뒤늦게 눈을 떠서 변수를 바라보니 이미 변수 값은 마지막 단어인 good day로 바뀌어 있는 것이다.

참고로 루프가 끝나서 변수가 사라져야(Scope 아웃) 마땅한 상황인데도 고루틴이 안전하게 값을 읽을 수 있는 이유는, Go 런타임이 똑똑하게 감시하다가 “어? 고루틴이 이 변수 아직 쓰고 있네?”하고 변수를 메모리 안전 구역(Heap)으로 대피시켜 주기 때문이다.

이 대참사를 해결하는 올바른 방법은, 루프를 돌 때마다 현재 단어의 ‘복사본’을 고루틴 함수에 매개변수로 직접 던져주는 것이다.

```
var wg sync.WaitGroup

for _, salutation := range []string{"hello", "greetings", "good day"} {
    wg.Add(1)

    // 인자로 문자열을 받도록 매개변수(salutation string)를 선언합니다.
    go func(salutation string) {
        defer wg.Done()
        fmt.Println(salutation) // 이제 안전하게 복사된 독립적인 값을 씁니다
    }(salutation) // 중요: 현재 루프 단계의 값을 복사해서 꼭 집어넣어 줍니다.
}

wg.Wait()
```

이렇게 짜야 우리가 원하는 대로 중복없이 세 단어가 무작위 순서로 예쁘게 잘 출력된다. 메모리 공간을 공유하는 건 강력하지만, 이런 동기화 문제를 항상 조심해야 한다.

(5) 고루틴의 무게 측정과 스케일

고루틴의 또 다른 압도적인 축복은 ‘상상을 초월할 정도로 가볍다’는 점이다. Go 공식 FAQ의 설명을 들어보자.

고루틴의 무게에 대한 Go의 설명

“새로 태어난 고루틴 하나가 사용하는 메모리가 고작 몇 킬로바이트(KB)에 불과합니다. 심지어 일하다가 용량이 부족해지면 런타임이 자동으로 메모리 스택 크기를 늘렸다 줄였다 관리해 줍니다. 덕분에 아주 작은 메모리 공간 안에서도 수십만 개의 고루틴이 동시에 살아 숨 쉴 수 있습니다.”

이 말이 진짜인지 코드로 직접 무게를 측정해보겠다.

참고로 고루틴은 중간에 갇혀서 안 끝나고 방치되면 프로그램이 꺼질 때까지 메모리에 영원히 살아남는 ‘고루틴 누수(Leak)’ 현상이 일어난다.

원래는 심각한 버그지만, 이번에는 고루틴의 무게를 재기 위해 일부러 영원히 안끝나는 고루틴 10,000개를 켜서 메모리 변화량을 측정해보겠다.

```
// [1] 현재 프로세스가 소모 중인 순수 하드웨어 메모리 바이트 수를 반환하는 헬퍼 함수
memConsumed := func() uint64 {
    runtime.GC() // 의도치 않은 쓰레기 메모리를 GC로 말끔히 청소하여 변수 통제
    var s runtime.MemStats
    runtime.ReadMemStats(&s)
    return s.Sys
}

var c <- chan interface{}
var wg sync.WaitGroup

// [2] 락에 걸려 영원히 탈출하지 못하고 메모리에 봉인될 무해한 좀비 함수 정의
noop := func() {wg.Done(); <- c}

const numGoroutines = 1e4 // [3] 대량의 법칙을 활용해 정확한 수치를 구하고자 10,000개
생산 결정

wg.Add(numGoroutines)

before := memConsumed() // 고루틴 생산 전 메모리 스냅샷 측정
for i := numGoroutines; i > 0; i--{
    go noop() // 10,000개의 고루틴을 폭발적으로 생산
}

wg.Wait()

after := memCousmed() // 고루틴 생산 후 메모리 스냅샷 측정
// 고루틴 1개당 평균 및 킬로바이트를 소모했는지 계산하여 출력
fmt.Printf("%.3fkb", float64(after-before)/numGoroutines/1024)
```

실행 결과:

2.817kb

정말 놀랍다. 고루틴 한 개당 고작 2.8KB 정도밖에 차지하지 않는다.

문서에 적힌 대로 몇 킬로바이트 수준이 맞았던 것이다.

이 계산대로라면, 우리가 쓰는 일반적인 컴퓨터 환경에서 메모리 용량별로 이론상 띄울 수 있는 고루틴의 최대 개수는 다음과 같다. (64비트 가상메모리 기준)

가용 메모리 (RAM)	동시 수용 가능한 고루틴 개수	크기 규모 (Order of Magnitude)
1 GB	약 371,800 개	수십만 단위
2 GB	약 743,600 개	수십만 단위
4 GB	약 1,487,300 개	백만 단위
8 GB	약 2,974,600 개	백만 단위
16 GB	약 5,949,200 개	백만 단위
32 GB	약 11,898,300 개	천만 단위

메모리가 8GB인 평범한 노트북에서도 이론상 수백만 개의 고루틴을 동시에 띄울 수 있다는 소리이다. 만약 고루틴이 아니라 옛날 방식의 OS 스레드였다면, 수천 개만 켜져도 컴퓨터 자원이 바닥나서 파랗게 질려 뺏어버렸을 것이다.

(6) 눈부시게 빠른 문맥 전환 성능

아무리 메모리를 적게 먹어도, 동시성 작업이 수만 개로 늘어나면 이 작업 저 작업으로 두뇌를 바꾸는 ‘문맥 전환’ 비용 때문에 컴퓨터가 버벅거리지 않을까 걱정이 될 수 있다.

OS 레벨에서 스레드끼리 자리를 바꿀 때는 레지스터 값, 메모리 맵, 록업 테이블 등 엄청나게 방대한 정보를 저장하고 새로 로드해야 하므로 비용이 꽤 많이 든다.

하지만 소프트웨어(Go 런타임) 수준에서 스케줄링을 관리하는 고루틴은, 딱 필요한 핵심 데이터만 골라서 저장하기 때문에 문맥 전환 비용이 압도적으로 저렴하다. 자신의 리눅스 노트북에서 직접 측정한 OS 스레드 대 고루틴의 문맥 전환 속도 비교 결과를 보여주겠다.

리눅스 실제 OS 스레드 간의 전환 속도 테스트

```
taskset -c 0 perf bench sched pipe -T
```

- 결과: 스레드 간에 메시지를 한번 주고받는 데 드는 시간은 대략 1.467 마이크로초였다.

Go 고루틴 간의 전환 속도 테스트(Benchmark)

```
// (작가가 작성한 고루틴 간 메시지 토스 속도 벤치마크 코드 실행)
BenchmarkContextSwitch    5000000    225 ns/op
```

- 결과: 고루틴 간의 문맥 전환 속도는 무려 225나노초가 나왔다.

나노초를 마이크로초로 환산하면 0.225마이크로초이다.

이 속도 차이만 보아도 알 수 있듯이, 고루틴 개수가 너무 많아져서 문맥 전환 때문에 프로그램이 멈추지 않을까 하는 걱정은 붙들어 매셔도 좋다.

고루틴의 상한선 한계는 여러분이 동시성 프로그램을 마음껏 짜는 데 있어 그 어떤 걸림돌도 되지 않는다.

(7) 결론

이제 여러분은 고루틴을 어떻게 시작하는지, 그리고 고루틴이 내부적으로 얼마나 가볍고 강력하게 작동하는지 완벽하게 이해했다.

앞서 2장에서 배운 암달의 법칙을 기억하나?

내 프로그램이 순차적인 영역에 가로막혀 있지 않다면, 고루틴을 많이 켜면 켜볼수록 컴퓨터의 멀티 프로세서 두뇌를 남김없이 활용하며 프로그램 성능이 무한하게 수평 확장된다.

고루틴은 컴퓨터 과학계의 축복이자 아주 값싸고 풍족한 자원이다.

그러니 앞으로 코드를 짤 때 비용 걱정은 저 멀리 던져주고, 동시성이 필요한 순간마다 아낌없이 go 키워드를 질러보자. 성능 문제를 논하는 건, 나중에 정밀 측정 결과 고루틴이 진짜 원인이라고 판명되었을 때 해도 늦지 않다.

[03-2] sync 패키지

(1) 개요

sync 패키지는 저수준의 메모리 접근 동기화에 가장 유용한 동시성 기본 도구(Primitive)들을 담고 있다. 만약 여러분이 기존에 멀티스레드나 메모리 공유 방식으로 동시성을 다루는 언어(Java, C++ 등)를 경험해 봤다면, 이 패키지에 들어있는 타입들이 매우 친숙하게 느껴질 것이다.

Go 언어가 다른 언어와 차별화되는 점은, 이러한 저수준의 메모리 동기화 도구들 위에 ‘채널’이라는 혁신적인 고수준 도구를 한 겹 더 얹어서 개발자에게 훨씬 넓은 선택지를 제공한다는 점이다.

“Go의 동시성 철학”에서 다루었듯이, 이 저수준 도구들은 주로 구조체 내부 같은 좁은 스코프(Scope)에서 제 역할을 다한다. 메모리 동기화 방식을 언제 쓸지는 온전히 여러분의 선택에 달려 있다.

그럼 sync 패키지가 자랑하는 대표적인 도구들을 하나씩 살펴보겠다.

(2) 대기 조율사(WaitGroup)

WaitGroup은 여러 동시성 작업들이 모두 끝날 때까지 메인 흐름을 효율적으로 붙잡아 두고 싶을 때 사용하는 도구이다. 단, 고루틴들이 돌고 나서 반환하는 결과물에는 관심이 없거나, 결과물을 수집할 다른 독립적인 통로가 이미 마련되어 있을 때만 써야 한다.

만약 고루틴의 결과물을 직접 받아야 한다면 WaitGroup 대신 채널과 select 문을 쓰는 것이 맞다.

WaitGroup이 고루틴의 종료를 어떻게 기다려주는지 전체 코드를 확인해보자.

```
package main

import (
    "fmt"
    "sync"
    "time"
)

func main() {
    var wg sync.WaitGroup

    wg.Add(1)
```

```

go func() {
    defer wg.Done()
    fmt.Println("1st goroutine sleeping...")
    time.Sleep(1 * time.Second)
}()

wg.Add(1)
go func() {
    defer wg.Done()
    fmt.Println("2nd goroutine sleeping...")
    time.Sleep(2 * time.Second)
}()

wg.Wait()
fmt.Println("All goroutines complete.")
}

```

위 코드를 실행하면 다음과 같은 결과가 출력된다.

```

2nd goroutine sleeping...
1st goroutine sleeping...
All goroutines complete.

```

WaitGroup의 핵심 메커니즘

WaitGroup은 여러 고루틴이 안전하게 공유할 수 있는 ‘동시성 보장 카운터’라고 생각하면 쉽다.

- Add(n): 카운터 숫자를 n만큼 더한다.(보통 일꾼을 투입하기 전에 부른다.)
- Done(): 카운터 숫자를 1만큼 뺀다. 내부적으로 Add(-1)을 호출한 것과 같다.
- Wait(): 카운터 숫자가 정확히 0이 될 때까지 하던 일을 멈추고 기다린다.

초보자가 반드시 저지르는 실수의 함정

위 코드에서 wg.Add(1) 호출문이 고루틴의 익명 함수 ‘바깥’에 있는 것에 주목하자.

만약에 Add(1)을 고루틴 자식의 익명 함수 ‘내부’에 집어 넣으면 어떻게 될까?

고루틴이 언제 스케줄링되어 실행될지는 아무도 보장할 수 없기 때문에,

자식 고루틴이 채 실행되기도 전에 메인 고루틴이 밑으로 혹은 내려가서 wg.Wait()을 먼저 만나 버릴 수 있다.

이때 카운터는 여전히 0이므로, Wait()은 자식을 전혀 기다리지 않고 그대로 프로그램을 종료시켜 버린다.(데이터 경쟁 버그 발생)

따라서 관례적으로 Add 호출은 그것이 추적할 고루틴의 생성 직전에 가장 가깝게 배치해야 한다. 만약 반복문(for)을 돌며 여러 고루틴을 띄울 때는 아래처럼 루프 직전에 한 방에 숫자를 더해두는 것이 아주 깔끔하다.

```
package main

import (
    "fmt"
    "sync"
)

func main() {
    hello := func(wg *sync.WaitGroup, id int) {
        defer wg.Done()
        fmt.Printf("Hello from %v!\n", id)
    }

    const numGreeters = 5
    var wg sync.WaitGroup

    wg.Add(numGreeters)
    for i := 0; i < numGreeters; i++ {
        go hello(&wg, i+1)
    }

    wg.Wait()
}
```

위 코드를 실행하면 다음과 같은 결과가 출력된다.

```
Hello from 5!
Hello from 4!
Hello from 3!
Hello from 2!
Hello from 1!
```

(3) 상호배제 자물쇠(Mutex and RWMutex)

기존 멀티스레드 언어에서 오신 분들이라면 Mutex라는 단어를 듣자마자 무릎을 탁 칠 것이다. 처음보더라도 전혀 겁먹을 필요없다. 개념은 정말 단순하다.

Mutex는 ‘Mutual Exclusion(상호배제)’의 줄임말로, 내 프로그램의 안전 구역인 ‘임계 영역’을 단단히 지키는 자물쇠이다.

1장에서 배웠듯이 공유 자원을 여러 일꾼이 동시에 건드리면 데이터가 엉망이 되므로, 한 번에 오직 한 놈만 들어오게 줄을 세우는 도구이다.

두 고루틴이 하나의 공용 숫자를 두고 더하고 빼는 연산을 할 때 Mutex를 사용하는 전체 코드를 보겠다.

```
package main

import (
    "fmt"
    "sync"
)

func main() {
    var count int
    var lock sync.Mutex

    increment := func() {
        lock.Lock()
        defer lock.Unlock()
        count++
        fmt.Printf("Incrementing: %d\n", count)
    }

    decrement := func() {
        lock.Lock()
        defer lock.Unlock()
        count--
        fmt.Printf("Decrementing: %d\n", count)
    }
}
```



```

var arithmetic sync.WaitGroup
for i := 0; i <= 5; i++ {
    arithmetic.Add(1)
    go func() {
        defer arithmetic.Done()
        increment()
    }()
}

for i := 0; i <= 5; i++ {
    arithmetic.Add(1)
    go func() {
        defer arithmetic.Done()
        decrement()
    }()
}

arithmetic.Wait()
fmt.Println("Arithmetic complete.")
}

```

위 코드를 실행하면 다음과 같은 결과가 출력된다.

```

Decrementing: -1
Incrementing: 0
Decrementing: -1
Incrementing: 0
Decrementing: -1
Decrementing: -2
Decrementing: -3
Incrementing: -2
Decrementing: -3
Incrementing: -2
Incrementing: -1
Incrementing: 0

```

Arithmetic complete.

- defer lock.Unlock() 패턴의 생활화

자물쇠를 열어주는 Unlock()은 언제나 에러가 나든 패닉이 터지든 무조건 실행되도록 defer 문맥에 묶어두는 것이 Go언어의 철칙이다. 이걸 빼먹으면 자물쇠가 영원히 잠겨서 프로그램이 통째로 고사하는 데드락 대참사가 터진다.

읽기/쓰기 분리 자물쇠(RWMutex)

임계 영역은 내 프로그램의 대표적인 성능 병목 구간이다. 자물쇠를 잠그고 푸는 행동 자체가 비용이 크기 때문에, 우리는 임계 영역 안에서 머무는 시간을 최소한으로 줄여야 한다.

이때 성능을 획득할 수 있는 아주 훌륭한 전략이 있다.

“공유 메모리를 여러 고루틴이 같이 쓰긴 하는데... 가만히 보니까 대부분의 일꾼은 값을 ‘읽기(Read)’만 하고, 값을 ‘쓰기(Write)’하는 일꾼은 아주 가끔만 오잖아?”라는 상황이다.

값을 읽기만 하는 일꾼들은 서로 데이터 경쟁을 일으키지 않으므로 굳이 한 명씩 줄 서서 읽을 필요가 없다. 다 같이 동시에 읽어도 안전하다.

이 영리한 조율을 가능하게 해주는 도구가 바로 sync.RWMutex이다.

- 읽기락(RLock): “나 값만 읽을 거야!”라고 선언한다. 누군가 값을 쓰고 있지만 않다면, 수백 수천 명의 읽기 고루틴이 자물쇠를 동시에 쥐고 임계 영역에 한 방에 들어갈 수 있다.

- 쓰기락(Lock): “나 값을 새로 수정할거야!”라고 선언한다. 전통적인 Mutex와 똑같이 작동하여, 읽는 사람이든, 쓰는 사람이든 아무도 못들어오게 혼자 완벽한 독점권을 가진다.

데이터를 수정하는 생성자 고루틴보다 데이터를 읽기만 하는 감시자 고루틴이 훨씬 더 많이 발생하는 경우에서 두 자물쇠의 성능 차이를 측정하는 전체 코드를 보겠다.

```
package main
```

```
import (
```

```
    "bytes"
```

```
    "fmt"
```

```
    "math"
```

```
    "os"
```

```

"sync"
"text/tabwriter"
"time"
)

func main() {
    producer := func(wg *sync.WaitGroup, l sync.Locker) {
        defer wg.Done()
        for i := 5; i > 0; i-- {
            l.Lock()
            l.Unlock()
            time.Sleep(1 * time.Nanosecond)
        }
    }

    observer := func(wg *sync.WaitGroup, l sync.Locker) {
        defer wg.Done()
        l.Lock()
        defer l.Unlock()
    }

    test := func(count int, mutex, rwMutex sync.Locker) time.Duration {
        var wg sync.WaitGroup
        wg.Add(count + 1)
        beginTestTime := time.Now()
        go producer(&wg, mutex)
        for i := count; i > 0; i-- {
            go observer(&wg, rwMutex)
        }
        wg.Wait()
        return time.Since(beginTestTime)
    }

    tw := tabwriter.NewWriter(os.Stdout, 0, 1, 2, ' ', 0)
    defer tw.Flush()
}

```

```

var m sync.RWMutex
fmt.Fprintf(tw, "ReadersWtRWMutexWtMutexWn")
for i := 0; i < 20; i++ {
    count := int(math.Pow(2, float64(i)))
    fmt.Fprintf(
        tw,
        "%dWt%vWt%vWn",
        count,
        test(count, &m, m.RLocker()),
        test(count, &m, &m),
    )
}
}

```

위 코드를 실행하며 다음과 같은 결과가 출력된다.

```

Readers RWMutex Mutex
1 38.343µs 15.854µs
2 21.86µs 13.2µs
4 31.01µs 31.358µs
8 63.835µs 24.584µs
16 52.451µs 78.153µs
32 75.569µs 69.492µs
64 141.708µs 163.43µs
128 176.35µs 157.143µs
256 234.808µs 237.182µs
512 262.186µs 434.625µs
1024 459.349µs 850.601µs
2048 840.753µs 1.663279ms
4096 1.683672ms 2.42148ms
8192 2.167814ms 4.13665ms
16384 4.973842ms 8.197173ms
32768 9.236067ms 16.247469ms
65536 16.767161ms 30.948295ms
131072 71.457282ms 62.203475ms

```

```
262144 158.76261ms 119.634601ms
524288 303.865661ms 231.072729ms
30.948ms (읽는 사람이 많아질수록 압도적인 격차)
```

위 결과표를 보면 읽는 사람(Readers)의 수가 적을 때는 일반 Mutex가 더 단순해서 미세하게 빠르기도 하지만, 읽는 고루틴 수가 수천 수만 개로 늘어나는 순간 RWMutex가 일반 자물쇠보다 무려 수배에서 수십 배 이상 빠른 성능을 보장해준다.

따라서 논리적으로 ‘읽기가 압도적으로 많은 구조’라면 일반 Mutex 대신 RWMutex를 선택하는 것이 언제나 현명한 설계이다.

(4) 조건 대기방(Cond)

sync.Cond 타입의 목적은 고루틴들이 어떤 특정 이벤트가 발생하기를 기다리거나, 혹은 사건이 터졌음을 사방에 알릴 때 만나는 만남의 광장이다.

여기서 말하는 이벤트란, 어떠한 세부 데이터도 담고 있지 않은 채 오직 어떤 사건이 방금 터졌다는 순수한 신호 그 자체를 뜻한다. 우리는 종종 고루틴이 일을 하다가 이 신호가 올 때까지 동작을 잠시 멈추고 효율적으로 대기하게 만들고 싶을 때가 있다. 만약 이 Cond 도구가 없다면 다음과 같이 무한 루프를 돌며 자원을 낭비해야 한다.

```
for conditionTrue() == false {
}
```

하지만 이 방식은 CPU 코어 하나를 통째로 낭비하게 된다. 이를 고치겠다고 중간에 슬립을 넣으면 조금은 나아지지만, 여전히 효율적이지 못하다.

```
for conditionTrue() == false {
    time.Sleep(1*time.Millisecond)
}
```

슬립 시간을 너무 길게 잡으면 프로그램 성능이 인위적으로 떨어지고, 너무 짧게 잡으면 CPU 연산량을 너무 많이 소모하게 된다.

가장 좋은 방법은 조건이 충족될 때까지 고루틴을 완전히 일시정지시켰다가, 신호가 올 때 깨어나 조건을 체크하게 만드는 것이다.

이것이 바로 `sync.Pool`과 `sync.Cond`가 해주는 일이다.

생성 비용이 무거운 객체를 동시성 프로그래밍에서 이 풀 패턴으로 다룰때, 시스템 전체의 처리 성능과 응답 속도가 얼마나 괴물처럼 도약할 수 있는지 완벽하게 입증되었다.

```
package main

import (
    "fmt"
    "sync"
)

func main() {
    var condition bool
    conditionTrue := func() bool { return condition }

    c := sync.NewCond(&sync.Mutex{})
    c.L.Lock()
    for conditionTrue() == false {
        c.Wait()
    }
    c.L.Unlock()
    fmt.Println("Condition met!")
}
```

Wait() 메서드가 가진 숨겨진 동작

`Cond.Wait()` 함수를 쓸 때는 내부에 숨겨진 동작을 기억해야 한다.

`Wait()`에 들어가는 순간 자기가 쥐고 있던 자물쇠를 자동으로 열어준다(`Unlock`).

그래야 다른 고루틴들이 상태를 바꿀 수 있기 때문이다.

그리고 신호를 받고 `Wait()`을 빠져나오는 순간 자동으로 자물쇠를 다시 잠근다(`Lock`).

코드를 볼 때는 마치 내가 락을 계속 쥔 채로 대기하고 있는 것처럼 착각하기 쉬우니 주의해야 한다.

신호보내기: Signal vs Broadcast

잠들어 있는 고루틴들을 깨우기 위해 Cond가 제공하는 메서드는 Signal과 Broadcast가 있다. 고정 길이2를 가진 대기열에서 두 메서드가 어떻게 소통을 조율하는지 전체 코드로 확인해 보겠다.

```
package main

import (
    "fmt"
    "sync"
    "time"
)

func main() {
    c := sync.NewCond(&sync.Mutex{})
    queue := make([]interface{}, 0, 10)

    removeFromQueue := func(delay time.Duration) {
        time.Sleep(delay)
        c.L.Lock()
        queue = queue[1:]
        fmt.Println("Removed from queue")
        c.L.Unlock()
        c.Signal()
    }

    for i := 0; i < 10; i++ {
        c.L.Lock()
        for len(queue) == 2 {
            c.Wait()
        }
        fmt.Println("Adding to queue")
        queue = append(queue, struct{}{})
        go removeFromQueue(1 * time.Second)
        c.L.Unlock()
    }
}
```

위 코드를 실행하면 다음과 같은 결과가 출력된다.

```
Adding to queue
Adding to queue
Removed from queue
Adding to queue
Removed from queue
Adding to queue
Removed from queue
Adding to queue
Removed from queue
Adding to queue
Removed from queue
Adding to queue
Removed from queue
Adding to queue
Removed from queue
Adding to queue
Removed from queue
Adding to queue
```

- Signal: 내부적으로 가장 오래 기다린 고루틴 딱 한 개만 콧 집어서 깨워준다.
- Broadcast: 현재 대기방 안에서 잠들어 있는 모든 고루틴에게 신호를 보내 전부 다 깨워버린다.

Broadcast는 특히 여러 시스템에 동시에 신호를 전달해야 할 때 유용하다. 마우스 버튼을 클릭했을 때, 이 클릭 이벤트를 구독하고 있는 수많은 고루틴 핸들러에게 동시에 알리는 상황에 완벽하게 들어맞는다.

```
package main

import (
    "fmt"
    "sync"
)
```



```

type Button struct {
    Clicked *sync.Cond
}

func main() {
    button := Button{Clicked: sync.NewCond(&sync.Mutex{})}

    subscribe := func(c *sync.Cond, fn func()) {
        var goroutineRunning sync.WaitGroup
        goroutineRunning.Add(1)
        go func() {
            goroutineRunning.Done()
            c.L.Lock()
            defer c.L.Unlock()
            c.Wait()
            fn()
        }()
        goroutineRunning.Wait()
    }

    var clickRegistered sync.WaitGroup
    clickRegistered.Add(3)

    subscribe(button.Clicked, func() {
        fmt.Println("Maximizing window.")
        clickRegistered.Done()
    })

    subscribe(button.Clicked, func() {
        fmt.Println("Displaying annoying dialog box!")
        clickRegistered.Done()
    })

    subscribe(button.Clicked, func() {
        fmt.Println("Mouse clicked.")
        clickRegistered.Done()
    })
}

```

```
button.Clicked.Broadcast()
clickRegistered.Wait()
}
```

위 코드를 실행하면 다음과 같은 결과가 출력된다.

```
Mouse clicked.
Maximizing window.
Displaying annoying dialog box!
```

하나의 신호로 수많은 대기자를 동시에 깨우는 행동은 일반 채널로는 구현하기가 무척 까다롭다. 이것이 `sync.Cond` 타입을 사용하는 핵심 이유이다.

(5) 단 한번의 실행 보장(Once)

여러 고루틴이 동시에 경쟁하며 함수를 호출할 때, 프로그램 전체에서 딱 한 번만 실행되도록 보장해야 할 때가 있다. 전체코드를 보자.

```
package main

import (
    "fmt"
    "sync"
)

func main() {
    var count int
    increment := func() {
        count++
    }

    var once sync.Once
    var increments sync.WaitGroup
    increments.Add(100)
```

```

    for i := 0; i < 100; i++ {
        go func() {
            defer increments.Done()
            once.Do(increment)
        }()
    }

    increments.Wait()
    fmt.Printf("Count is %d\n", count)
}

```

위 코드를 실행하면 다음과 같은 결과가 출력된다.

```
Count is 1
```

sync.Once는 다른 고루틴에서 호출하더라도 Do 메서드에 전달된 함수가 단 한번만 실행되도록 보장한다. Go의 표준 라이브러리 소스코드 내에서도 이 패턴이 얼마나 자주 쓰이는지 확인할 수 있다.

```
grep -ir sync.Once $(go env GOROOT)/src | wc -l
```

```
70
```

sync.Once를 쓸 때 주의할 점

sync.Once는 전달되는 함수의 종류를 구별하지 않고 Do가 호출된 회수만 체크한다. 아래 코드를 보자.

```

package main

import (
    "fmt"
    "sync"
)

func main() {
    var count int

```

```

increment := func() { count++ }
decrement := func() { count-- }

var once sync.Once
once.Do(increment)
once.Do(decrement)

fmt.Printf("Count: %d\n", count)
}

```

위 코드를 실행하면 다음과 같은 결과가 출력된다.

```
Count: 1
```

두 번째 호출한 decrement 함수는 실행되지 않는다. 따라서 sync.Once 변수 하나당 관리할 함수 하나를 아주 좁은 스코프(구조체 내부 등)에 단단히 묶어두는 것이 안전하다.

또한 다음 예시처럼 순환 참조 구조로 잘못 설계하면 교착 상태가 터질 수 있으니 주의해야 한다.

```

package main

import (
    "sync"
)

func main() {
    var onceA, onceB sync.Once
    var initB func()
    initA := func() { onceB.Do(initB) }
    initB = func() { onceA.Do(initA) }

    onceA.Do(initA) // 데드락 발생으로 프로그램이 멈춤
}

```

(6) 객체 재활용 창고(Pool)

sync.Pool은 동시성 환경에서 안전하게 사용할 수 있는 객체 재활용 보관소이다.

생성 비용이 비싼 자원(예: 데이터베이스 연결, 대형 메모리 버퍼 등)의 생성을 제한하면서, 수많은 고루틴의 접근을 허용하고 싶을 때 사용한다.

- Get(): 창고에 재활용 가능한 객체가 있는지 확인하고 있으면 반환한다.
창고가 비어있다면 우리가 정의해둔 New 함수를 실행해서 새 객체를 만들어 반환한다.
- Put(x): 사용이 끝난 객체 x를 다음 프로세스들이 재사용할 수 있도록 창고에 다시 반납한다.

```
package main

import (
    "fmt"
    "sync"
)

func main() {
    myPool := &sync.Pool{
        New: func() interface{} {
            fmt.Println("Creating new instance.")
            return struct{}{}
        },
    }

    myPool.Get()
    instance := myPool.Get()
    myPool.Put(instance)
    myPool.Get()
}
```

위 코드를 실행하면 다음과 같은 결과가 출력된다.

```
Creating new instance.
Creating new instance.
```

창고에 재고가 채워질 세 번째 Get() 호출 시점에는 New 함수를 실행하지 않고 자원을 재활용하는 것을 볼 수 있다.

Go 언어에는 가비지 컬렉터가 있어서 자동으로 메모리를 치워주는데도 왜 굳이 풀을 써서 재활용할까? 1,024바이트(1KB)짜리 메모리 버퍼를 동시에 무려 100만 번 이상 켜서 사용하는 극단적인 환경의 코드를 보자.

```
package main

import (
    "fmt"
    "sync"
)

func main() {
    var numCalcsCreated int
    calcPool := &sync.Pool{
        New: func() interface{} {
            numCalcsCreated += 1
            mem := make([]byte, 1024)
            return &mem
        },
    }

    calcPool.Put(calcPool.New())
    calcPool.Put(calcPool.New())
    calcPool.Put(calcPool.New())
    calcPool.Put(calcPool.New())

    const numWorkers = 1024 * 1024
    var wg sync.WaitGroup
    wg.Add(numWorkers)
    for i := numWorkers; i > 0; i-- {
        go func() {
            defer wg.Done()
            mem := calcPool.Get().(*[]byte)
```

```

        defer calcPool.Put(mem)
    }()
}
wg.Wait()
fmt.Printf("%d calculators were created.", numCalcsCreated)
}

```

위 코드를 실행하면 다음과 같은 결과가 출력된다.

```
8 calculators were created.
```

매번 새 메모리를 할당했다면 수 기가바이트의 자원을 소모하며 가비지 컬렉터에 엄청난 부담을 주었겠지만, 단 8개의 객체만 생성되어 대량의 작업을 완벽하게 재활용한 것을 볼 수 있다.

실무 예시: 네트워크 데몬의 응답 최적화

또 다른 흔한 활용처는 대용량 트래픽을 처리하는 고성능 네트워크 서버를 구축할 때이다. 사용자가 접속할 때마다 가상의 무거운 커넥션을 새로 생성하는 서버를 먼저 테스트해 보겠다.

```

package main
import (
    "fmt"
    "io/ioutil"
    "log"
    "net"
    "sync"
    "testing"
    "time"
)

func connectToService() interface{} {
    time.Sleep(1 * time.Second)
    return struct{}{}
}

```

```

func startNetworkDaemon() *sync.WaitGroup {
    var wg sync.WaitGroup
    wg.Add(1)
    go func() {
        server, err := net.Listen("tcp", "localhost:8080")
        if err != nil {
            log.Fatalf("cannot listen: %v", err)
        }
        defer server.Close()
        wg.Done()
        for {
            conn, err := server.Accept()
            if err != nil {
                log.Printf("cannot accept connection: %v", err)
                continue
            }
            connectToService()
            fmt.Fprintln(conn, "")
            conn.Close()
        }
    }()
    return &wg
}

func main() {
    daemonStarted := startNetworkDaemon()
    daemonStarted.Wait()

    res := testing.Benchmark(func(b *testing.B) {
        for i := 0; i < b.N; i++ {
            conn, err := net.Dial("tcp", "localhost:8080")
            if err != nil {
                b.Fatalf("cannot dial host: %v", err)
            }
        }
    })
}

```



```

        if _, err := ioutil.ReadAll(conn); err != nil {
            b.Fatalf("cannot read: %v", err)
        }
        conn.Close()
    }
})
fmt.Println(res)
}

```

위 벤치마크 테스트 결과는 대략 다음과 같이 나온다.

10	1000385643 ns/op
----	------------------

접속 한 번을 처리하는 데 약 1초가 소요된다. 이제 자원을 미리 풀에 채워두고 활용하는 최적화 구조로 변경하여 검증해보겠다.

```

package main

import (
    "fmt"
    "io/ioutil"
    "log"
    "net"
    "sync"
    "testing"
    "time"
)

func connectToService() interface{} {
    time.Sleep(1 * time.Second)
    return struct{}{}
}

func warmServiceConnCache() *sync.Pool {
    p := &sync.Pool{
        New: connectToService,
    }
}

```

```

    }
    for i := 0; i < 10; i++ {
        p.Put(p.New())
    }
    return p
}

func startNetworkDaemon() *sync.WaitGroup {
    var wg sync.WaitGroup
    wg.Add(1)
    go func() {
        connPool := warmServiceConnCache()
        server, err := net.Listen("tcp", "localhost:8080")
        if err != nil {
            log.Fatalf("cannot listen: %v", err)
        }
        defer server.Close()
        wg.Done()
        for {
            conn, err := server.Accept()
            if err != nil {
                log.Printf("cannot accept connection: %v", err)
                continue
            }
            svcConn := connPool.Get()
            fmt.Fprintln(conn, "")
            connPool.Put(svcConn)
            conn.Close()
        }
    }()
    return &wg
}

func main() {
    daemonStarted := startNetworkDaemon()

```

```

daemonStarted.Wait()

res := testing.Benchmark(func(b *testing.B) {
    for i := 0; i < b.N; i++ {
        conn, err := net.Dial("tcp", "localhost:8080")
        if err != nil {
            b.Fatalf("cannot dial host: %v", err)
        }
        if _, err := ioutil.ReadAll(conn); err != nil {
            b.Fatalf("cannot read: %v", err)
        }
        conn.Close()
    }
})
fmt.Println(res)
}

```

위 벤치마크 테스트 결과는 다음과 같이 나온다.

```
5000 2904307 ns/op
```

자원을 매번 새로 생성하는 대신 sync.Pool에서 꺼내 썼더니,
응답 속도가 약 1,000배(세 자릿수 자릿수)나 폭발적으로 빨라졌다.

단, 풀에 보관할 객체들은 무조건 형태가 균일해야 한다. 어떤 놈은 크기가 작고 어떤놈은 크기가 큰 가변 슬라이스 같은 것들을 풀에 넣으면, 꺼냈을 때 내가 원하는 크기일 확률이 낮아져서 이를 다시 재할당하고 형변환하는 비용이 오히려 더 커져 손해를 볼 수 있다.

sync.Pool 설계 핵심 요약

- sync.Pool 을 생성할 때 설정하는 New 변수는 멀티 고루틴 환경에서 호출되어도 안전해야 (Thread-safe)한다.
- Get()으로 꺼낸 객체의 내부 데이터 상태를 절대로 신뢰해서는 안된다. (과거 쓰던 찌꺼기가 남아 있을 수 있으므로 반드시 사용 전 초기화해야 한다.)
- 사용이 끝나면 반드시 Put()을 호출해 풀에 반납해야 한다. 일반적으로 defer를 사용한다.
- 풀 내부의 객체들은 모양과 성격이 일정해야 효과가 극대화된다.

[03-3] 채널

토니 호어 박사의 CSP이론을 Go언어의 심장부에 직접 이식한 핵심 도구인 채널에 대해 알아보겠다.

(1) 채널의 기본 개념과 생성

채널을 마치 거대한 정보의 강줄기나 데이터를 실어 나르는 파이프라인과 같다.

상류(Upstream)에서 데이터를 채널에 밀어 넣으면, 하류(Downstream)에 있는 다른 고루틴이 그 데이터를 읽어가는 구조이다.

이 때문에 Go 개발자들은 채널 변수 이름을 지을 때 뒤에 Stream이라는 접미사를 자주 붙이곤 한다. 채널을 쓰면 프로그램의 각 파트들이 서로의 존재를 전혀 몰라도, 오직 동일한 채널의 메모리 주소만 참조하고 있다면 완벽하게 소통할 수 있다.

채널을 선언하고 생성하는 전체 코드는 다음과 같다.

```
package main

import "fmt"

func main() {
    // 1. 채널 변수 선언 (빈 인터페이스 타입을 담는 채널)
    var dataStream chan interface{}

    // 2. 내장 make 함수를 사용해 채널 인스턴스 실제 생성
    dataStream = make(chan interface{})

    // 실제 실무에서는 단축 표기법(:=)으로 생성하는 것이 관례입니다.
    intStream := make(chan int)

    fmt.Printf("dataStream type: %TWn", dataStream)
    fmt.Printf("intStream type: %TWn", intStream)
}
```

위 코드를 실행하면 다음과 같은 결과가 출력된다.

```
dataStream type: chan interface {}  
intStream type: chan int
```

dataStream은 interface{}(빈 인터페이스) 타입으로 만들었기 때문에 어떤 종류의 데이터든 제한 없이 넣고 뺄 수 있다. 반면 intStream은 오직 정수형(int) 데이터만 통과할 수 있도록 엄격하게 타입을 제한한다.

(2) 단방향 채널의 규칙과 타입 안정성

기본적으로 생성한 채널은 데이터를 보낼 수도 있고 받을 수도 있는 양방향 채널이다. 하지만 Go의 강력한 타입 시스템을 활용하면 오직 데이터를 보내기만 하거나, 받기만 할 수 있는 단방향 채널을 선언할 수 있다. 화살표(<-)의 위치에 따라 성격이 결정된다.

```
package main  
  
import "fmt"  
  
func main() {  
    // 수신 전용 채널 (채널에서 데이터가 밖으로 나오는 모양)  
    var receiveChan <-chan interface{}  
  
    // 송신 전용 채널 (채널 안으로 데이터가 들어가는 모양)  
    var sendChan chan<- interface{}  
  
    // 양방향 채널 생성  
    dataStream := make(chan interface{})  
  
    // 양방향 채널을 단방향 채널 변수에 대입 (컴파일러가 자동 변환 지원)  
    receiveChan = dataStream  
    sendChan = dataStream  
  
    fmt.Printf("receiveChan type: %TWn", receiveChan)  
    fmt.Printf("sendChan type: %TWn", sendChan)  
}
```

위 코드를 실행하면 다음과 같은 결과가 출력된다.

```
receiveChan type: <-chan interface {}  
sendChan type: chan<- interface {}
```

만약 송신 전용 채널에서 데이터를 읽으려고 하거나, 수신 전용 채널에 데이터를 넣으려고 하면 Go컴파일러를 발생시켜 빌드를 가로막는다.

잘못 구현된 아래의 코드를 컴파일하겠다.

```
package main  
  
func main() {  
    writeStream := make(chan<- interface{})  
    readStream := make(<-chan interface{})  
  
    // 에러 발생: 송신 전용 채널에서 값을 빼려고 함  
    <-writeStream  
  
    // 에러 발생: 수신 전용 채널에 값을 넣으려고 함  
    readStream <- struct{}{}  
}
```

이 코드를 빌드하려고 하면 다음과 같은 컴파일 에러가 출력된다.

```
invalid operation: <-writeStream (receive from send-only type chan<- interface {})  
invalid operation: readStream <- struct {} literal (send to receive-only type <-  
chan interface {})
```

이 단방향 채널 스펙 덕분에 우리는 API나 함수를 설계할 때 이 함수는 이 채널의 데이터를 오직 소비(Read)만 하는 안전한 함수라는 사실을 코드 단에서 보장할 수 있다.

(3) 채널의 동기화 메커니즘과 데드락

고루틴 파트에서 자식 고루틴을 띄워놓고 메인 함수가 기다려주지 않으면 자식이 실행되기도 전에 프로그램이 끝난다고 배웠다.

하지만 아래 코드는 별도의 WaitGroup 조율사가 없는데도 정상 작동한다.

전체 코드를 확인해보자.

```
package main

import "fmt"

func main() {
    stringStream := make(chan string)

    go func() {
        // 익명 고루틴이 채널에 문자열을 밀어 넣음
        stringStream <- "Hello channels!"
    }()

    // 메인 고루틴이 채널에서 데이터를 꺼내 출력함
    fmt.Println(<-stringStream)
}
```

위 코드를 실행하면 다음과 같은 결과가 출력된다.

```
Hello channels!
```

이 코드가 정상적으로 동기화되는 이유는 Go언어의 채널이 기본적으로 블로킹(대기 체계) 방식으로 작동하기 때문이다.

- 채널에 데이터 채우기(`ch <- data`): 채널이 가득 차 있다면, 누군가 그 채널에서 데이터를 한 개 빼내서 빈 자리를 만들어 줄 때까지 데이터를 넣으려는 고루틴은 그자리에서 멈춰 서서 대기한다.

- 채널에서 데이터 빼기(`<-ch`): 채널이 텅 비어 있다면, 누군가 그 채널에 데이터를 하나라도 던져줄 때까지 데이터를 꺼내려는 고루틴은 그 자리에 멈춰 서서 대기한다.

위 코드에서 `fmt.Println(<-stringStream)` 문장은 채널에 데이터가 들어올 때까지 메인 흐름을 강제로 붙잡아 두는 확실한 조인 포인트 역할을 수행한다.

덕분에 프로그램이 조기 종료되지 않는다.

하지만 프로그램 구조를 잘못 작성하면 이 대기 법칙 때문에 프로그램이 영원히 마비되는 데드락(Deadlock)이 발생한다.

```
package main

import "fmt"

func main() {
    stringStream := make(chan string)

    go func() {
        if 0 != 1 {
            return // 채널에 아무 값도 안 넣고 고루틴을 그냥 종료시켜 버림
        }
        stringStream <- "Hello channels!"
    }()

    // 대참사 발생: 메인 고루틴은 영원히 오지 않을 데이터를 기다리며 얼어붙습니다.
    fmt.Println(<-stringStream)
}
```

위 코드를 실행하면 다음과 같은 런타임 패닉 결과가 출력된다.

```
fatal error: all goroutines are asleep - deadlock!

goroutine 1 [chan receive]:
main.main()
    /tmp/main.go:15 +0x97
```

자식 일꾼은 조건문에 의해 데이터를 넣지 않고 조기 종료되어 사라졌는데, 메인 함수는 텅 빈 채널만 바라보며 평생 대기 상태에 빠진다.

Go 런타임은 모든 고루틴이 잠자는 상태에 빠졌음을 감지하고 데드락 패닉 에러를 터트리며 시스템을 강제 종료한다.

(4) 채널 닫기와 복수 반환값 활용

데이터 전송이 완전히 끝났을 때 하류에 있는 고루틴들에게 이제 더 이상 줄 데이터가 없으니 하던 일 마무리하라고 신호를 보내야 할 때가 있다. 이때 사용하는 키워드가 바로 `close`이다.

채널에서 데이터를 읽을 때, 사실은 화살표 인자가 두 개의 결과값을 반환할 수 있다.
전체 코드를 보겠다.

```
package main

import "fmt"

func main() {
    stringStream := make(chan string)

    go func() {
        stringStream <- "Hello channels!"
    }()

    // 두 번째 불리언 반환값(ok)을 같이 받습니다.
    salutation, ok := <-stringStream
    fmt.Printf("(%v): %v\n", ok, salutation)
}
```

위 코드를 실행하면 다음과 같은 결과가 출력된다.

```
(true): Hello channels!
```

이 두 번째 반환값인 `ok` 변수는 무엇을 뜻할까?

- `true`: 방금 읽은 데이터가 누군가 정상적으로 채널에 써서 전달해 준 진짜 데이터라는 뜻
- `false`: 상류에서 채널을 닫아버렸기 때문에(`close`) 더 이상 데이터가 없어서 채널이 뿔어진 기본 기본값이라는 뜻

Go 언어의 채널은 닫힌 이후에도 언제든지, 심지어 무한대로 데이터를 계속 꺼내 읽을 수 있다. 닫힌 채널을 읽으면 어떤일이 벌어지는지 전체 코드로 확인하자.

```
package main

import "fmt"

func main() {
    intStream := make(chan int)
    close(intStream) // 생성하자마자 바로 채널을 닫아버림

    // 닫힌 채널에서 값을 두 번 연속 꺼내 읽어봅니다.
    integer1, ok1 := <-intStream
    fmt.Printf("(%v): %vWn", ok1, integer1)

    integer2, ok2 := <-intStream
    fmt.Printf("(%v): %vWn", ok2, integer2)
}
```

위 코드를 실행하면 다음과 같은 결과가 출력된다.

```
(false): 0
(false): 0
```

채널에 값을 단 한 번도 넣은 적이 없는데도 에러가 나지 않고 값이 계속 읽힌다. 대신 ok가 false를 반환하면, 값은 int 타입의 기본 재료값인 0을 뱉어낸다. 이 메커니즘이 존재하는 이유는 하나의 상류 생산자가 채널을 닫았을 때, 하류에 있는 수 많은 소비자 고루틴들이 남김없이 안전하게 데이터를 다 읽어갈 수 있도록 보장하기 위함이다.

(5) 채널 순회 패턴과 다중 고루틴 동시 깨우기

닫힌 채널이 false를 뱉는 성질을 이용하면 두 가지 아주 세련된 동시성 패턴을 구사할 수 있다.

for range 문을 통한 채널 자동 순회

for range문에 채널을 집어넣으면 채널에 데이터가 들어올 때마다 자동으로 루프를 돌며 꺼내오고, 채널이 닫히면 알아서 루프를 탈출하는 훌륭한 자동 시스템이 구축된다.

전체 코드를 확인하자.

```
package main

import "fmt"

func main() {
    intStream := make(chan int)

    go func() {
        defer close(intStream) // 고루틴이 끝날 때 채널을 닫아줍니다.
        for i := 1; i <= 5; i++ {
            intStream <- i
        }
    }()

    // 채널에 데이터가 올 때마다 자동으로 하나씩 쏙쏙 꺼내옵니다.
    for integer := range intStream {
        fmt.Printf("%v ", integer)
    }
    fmt.Println()
}
```

위 코드를 실행하면 다음과 같은 결과가 출력된다.

```
1 2 3 4 5
```

for range 문을 쓰면 두 번째 반환값(ok)을 검사해서 break를 거는 수동 코드를 짜지 않아도, 런타임이 알아서 닫힘 상태를 감지하고 안전하게 반복문을 끝내준다.

채널 닫기를 이용한 전원 동시 깨우기(Broadcast)

만약 여러 고루틴이 하나의 채널을 바라보며 대기하고 있을 때, 이 고루틴들을 한 방에 전원 기상시키고 채널에 값을 일일이 짜주는 것보다 채널을 단 한번 close 해버리는 것이 훨씬 빠르고 효율적이다. 닫힌 채널은 누구나 제한 없이 즉시 읽어갈 수 있기 때문이다.

전체 코드를 확인하자.

```
package main

import (
    "fmt"
    "sync"
)

func main() {
    begin := make(chan interface{})
    var wg sync.WaitGroup

    // 5명의 일꾼 고루틴을 대기시킵니다.
    for i := 0; i < 5; i++ {
        wg.Add(1)
        go func(id int) {
            defer wg.Done()
            <-begin // 모든 고루틴이 출발 신호 채널이 열릴 때까지 여기서 대기
                합니다.

            fmt.Printf("%v has begun\n", id)
        }(i)
    }

    fmt.Println("Unblocking goroutines...")
    close(begin) // 채널을 닫아서 대기하던 모든 고루틴을 동시에 깨웁니다.
    wg.Wait()
}
```

위 코드를 실행하면 다음과 같다.

```
Unblocking goroutines...
4 has begun
2 has begun
3 has begun
0 has begun
1 has begun
```

이전 섹션에서 이 행동을 하려고 `sync.Cond` 라는 복잡한 대기방 도구를 배웠지만, 사실 Go언어에서는 이처럼 채널 닫기 패턴을 사용하면 훨씬 직관적이고 다채롭게 조립 가능한 동시 깨우기를 구현할 수 있다.

(6) 버퍼를 가진 채널의 작동 원리

채널을 만들 때 두 번째 인자로 숫자를 채워 넣으면 버퍼를 가진 채널이 탄생한다.

버퍼란

채널 내부에 마련된 간이 보관용 슬롯(대기 창고)이라고 생각하면 된다.

```
package main

import "fmt"

func main() {
    // 4개의 보관 슬롯을 가진 버퍼 채널 생성
    dataStream := make(chan interface{}, 4)
    fmt.Printf("dataStream cap: %d\n", cap(dataStream))
}
```

위 코드를 실행하면 다음과 같은 결과가 출력된다.

```
dataStream cap: 4
```

버퍼 크기가 4라는 것은, 하류에서 데이터를 꺼내 읽어주는 사람이 아무도 없더라도 혼자서 연속으로 4번까지는 대기 없이 데이터를 채워 넣을 수 있다는 뜻이다.

우리가 지금까지 썼던 일반 채널은 사실 버퍼 크기가 0인 채널(`make(chan int, 0)`)과 같다. 즉, 넣자마자 바로 받아주는 사람이 대기하고 있어야만 전송이 성공하는 비버퍼형 채널이었던 것이다.

버퍼 채널 내부에서 데이터가 어떻게 채워지고 막히는지 실제 연산 흐름을 시각적으로 구현한 전체 코드를 통해 마스터해보자.

생산자가 보낼 데이터의 총 개수를 정확히 알고 있을 때, 그 크기만큼 딱 맞춰 버퍼 창고를 준비해 속도 저하 없이 털어 넣는 최적화 기법이다.

```
package main

import (
    "bytes"
    "fmt"
    "os"
)

func main() {
    var stdoutBuff bytes.Buffer
    defer stdoutBuff.WriteTo(os.Stdout)

    // 생산자가 5개의 데이터를 보낼 것을 알기에 4칸의 버퍼 채널을 생성
    intStream := make(chan int, 4)

    go func() {
        defer close(intStream)
        defer fmt.Fprintln(&stdoutBuff, "Producer Done.")
        for i := 0; i < 5; i++ {
            fmt.Fprintf(&stdoutBuff, "Sending: %d\n", i)
            intStream <- i // 버퍼 덕분에 소비자가 안 읽어가도 연속으로 채
            // 넣을 수 있습니다.
        }
    }()

    // 하류 소비자는 상류 고루틴이 다 일하고 퇴근한 뒤에 천천히 데이터를 읽어옵니다.
    for integer := range intStream {
```

```
        fmt.Fprintf(&stdoutBuff, "Received %v.\n", integer)
    }
}
```

위 코드를 실행하면 다음과 같은 결과가 출력된다.

```
Sending: 0
Sending: 1
Sending: 2
Sending: 3
Sending: 4
Producer Done.
Received 0.
Received 1.
Received 2.
Received 3.
Received 4.
```

결과 창을 보면 상류 익명 고루틴이 하류 메인 함수가 데이터를 단 한개도 꺼내 가기 전에 혼자서 5개의 전송 작업을 순식간에 마치고 “Producer Done.”을 외치며 퇴근한 것을 볼 수 있다. 특정한 대용량 연산 환경에서 아주 유용한 최적화 기법이다.

버퍼 채널은 고루틴 간의 속도 차이를 조율해 주는 고마운 보관소 역할을 하지만, 실무에서는 신중하게 도입해야 한다.

대기 창고를 넉넉하게 주면 당장 막히는 현상이 사라지기 때문에 성능이 좋아진 것처럼 착각하기 쉽지만, 이는 진짜 데드락 버그를 일시적으로 눈앞에서 가려주는 위험한 마취제가 될 수 있다.

(7) Nil 채널의 패닉과 동작 상태 총정리

채널을 선언만하고 make로 초기화하지 않으면 그 채널의 값은 nil이 된다.

nil 채널에서 데이터를 꺼내려고 하거나(<-dataStream), 데이터를 넣으려고 하면 (dataStrea <- strcut){} 프로그램은 자리에 얼어붙어 영원히 대기하는 블로킹 상태에 빠진다. 더욱 위험한 것은 nil 채널을 닫을 때이다. 전체 코드로 예외 상황을 확인해 보자.

```
package main

import "fmt"

func main() {
    var dataStream chan interface{}
    fmt.Println("Closing nil channel...")
    close(dataStream) // 예외 발생: 초기화되지 않은 nil 채널을 닫으려고 함
}
```

위 코드를 실행하면 다음과 같은 런타임 패닉 결과가 출력되며 시스템이 강제 종료된다.

```
Closing nil channel...
panic: close of nil channel

goroutine 1 [running]:
main.main()
    /tmp/main.go:8 +0x51
```

nil 채널을 닫아버리면 즉시 시스템이 즉사하므로, 채널을 다룰 때는 항상 make로 초기화가 완료되었는지 철저하게 확인해야 한다.

지금까지 배운 채널의 모든 상태별 연산 결과는 다음과 같다.

채널의 상태	읽기 연산 (<-ch)	쓰기 연산 (ch <- 값)	닫기 연산 (close(ch))
nil 상태	영원히 대기 (Block)	영원히 대기 (Block)	패닉 (Panic) 발생
열려있고 데이터 있음	즉시 값을 꺼냄	공간 있으면 성공 / 꽉 차면 대기	정상적으로 채널을 닫음
열려있고 창고 빔	데이터 올 때까지 대기	즉시 성공	정상적으로 채널을 닫음
이미 닫힌 상태 (Closed)	제로값 반환 (ok == false)	패닉 (Panic) 발생	패닉 (Panic) 발생
수신 전용 채널	정상 작동	컴파일 에러 (Build Fail)	컴파일 에러 (Build Fail)
송신 전용 채널	컴파일 에러 (Build Fail)	정상 작동	정상 작동

(8) 채널 소유권 설계 원칙

표를 보면 알겠지만 채널은 잘못 건드리면 고루틴이 대책없이 멈추거나 시스템이 패닉을 일으키는 등 위험 요소를 가지고 있다.

이 위험천만한 도구를 안전하게 다루기 위해 Go 개발자가 제안하는 최고의 마스터 원칙은 바로 채널 소유권을 정하는 것이다.

- 채널 소유권 고루틴의 4대 의무

1. 채널을 직접 생성한다.
2. 채널에 데이터를 안전하게 채워넣거나(Write), 소유권을 다른 곳으로 넘긴다.
3. 일이 끝나면 채널을 확실하게 닫아준다(close).
4. 이 1, 2, 3번 과정을 하나의 함수 내부에 캡슐화해서 외부에는 수신 전용 채널(<-chan)로만 노출한다.

이 규칙을 준수하여 작성한 전체 코드를 확인해보자.

```
package main

import "fmt"

func main() {
    // 1. 채널의 소유권을 가진 생산자 함수 정의
    chanOwner := func() <-chan int {
        resultStream := make(chan int, 5) // 직접 생성 (의무 1)

        go func() {
            defer close(resultStream) // 일이 끝나면 확실히 닫음 (의무 3)
            for i := 0; i <= 5; i++ {
                resultStream <- i // 데이터 채워 넣음 (의무 2)
            }
        }()

        return resultStream // 수신 전용 채널로 수출! (의무 4)
    }

    // 2. 소비자는 그저 소유자가 안전하게 만들어 준 채널을 가져다 쓰기만 하면 됩니다.
    resultStream := chanOwner()
    for result := range resultStream {
        fmt.Printf("Received: %d\n", result)
    }
    fmt.Println("Done receiving!")
}
```

위 코드를 실행하면 다음과 같은 결과가 출력된다.

```
Received: 0
Received: 1
Received: 2
Received: 3
Received: 4
Received: 5
Done receiving!
```

소유권 분리가 가져다주는 장점

- 채널을 소유자가 함수 내부에서 자기가 생성하므로, 실수로 nil 채널에 값을 넣거나 닫아서 생기는 버그가 원천 차단된다.
- 언제 닫아야 할지 소유자가 완벽하게 결정하므로, 이미 닫힌 채널에 또 값을 넣거나 닫아서 발생하는 대형 패닉 에러가 예방된다.
- 데이터를 소비하는 유저 입장에서는 복잡한 자물쇠나 닫기 연산을 고민할 필요 없이, 오직 데이터가 올 때 안전하게 대기하고 채널이 닫히면 탈출한다는 본연의 임무에만 충실할 수 있다.

채널의 소유권 범위를 언제나 눈에 보일 정도로 최대한 작고 맑게 유지해야 한다.

만약 거대한 구조체 변수 여기저기서 하나의 채널을 공유하고 심지어 멤버 메서드 수십 개가 그 채널을 난잡하게 건드리기 시작하면, 그 프로그램은 머지 않아 동시성 버그의 지옥으로 변하게 될 것이다.

채널은 고루틴들을 서로 단단하고 아름답게 묶어주는 최고의 접착제이다.

[03-4] select

Go언어에서 동시성 제어의 꽃이자 채널들을 하나로 묶어 거대한 흐름을 만들어내는 select에 대해 알아보겠다.

이전 섹션에서 채널이 고루틴들을 서로 연결하는 접착제라고 했다면, select 문은 그 채널들을 한 곳에 모아 제어하는 컨트롤 타워와 같다.

select 문은 단일 함수 내에서 채널들을 조율할 때도 쓰이지만, 시스템 전체의 여러 컴포넌트가 만나는 교차점에서도 강력한 역할을 수행한다.

이 중요한 길목에서 select 문은 취소(Cancellation), 타임아웃(Timeout), 대기(Waiting), 기본값(Default) 등의 개념을 채널과 안전하게 결합해 준다.

제시해주신 세 가지 원칙(시각적인 이모티콘 사용 금지, 생략 없는 전체 코드 작성, 출력 결과 명시)을 엄격히 준수하여 정리하겠다.

(1) select 문의 기본 구조

select 문의 문법은 기존의 switch 문과 겉모습이 매우 닮았다.

switch 문처럼 여러 개의 case문을 나열하고 조건을 감싸는 형태를 취한다.

하지만 유사점은 딱 거기까지이다.

switch 문과 달리, select 문의 case들을 위에서부터 순서대로 조건을 검사하지 않으며, 일치하는 조건이 없다고 해서 아래로 자연스럽게 흘러내려가는 구조도 아니다.

```
package main

import "fmt"

func main() {
    var c1, c2 <-chan interface{}
    var c3 chan<- interface{}

    // 이 코드는 예시 구조를 보여주기 위한 것으로, 모든 채널이 nil이므로 실행 시 블로킹됩니다.
    // 실제 환경에서 어떻게 배치되는지 직관적인 형태를 확인해 보세요.
    fmt.Println("select statement structure:")
    select {
    case <-c1:
```

```

// c1 채널에서 데이터를 읽을 수 있을 때 실행
case <-c2:
// c2 채널에서 데이터를 읽을 수 있을 때 실행
case c3 <- struct{}{}:
// c3 채널에 데이터를 공간이 있어서 넣을 수 있을 때 실행
}
}

```

select 블록이 실행되면 내부에 나열된 모든 채널의 읽기와 쓰기 연산이 동시에 고려된다.

- 읽기 연산의 경우: 채널에 데이터가 차 있거나 채널이 닫혔는지(close)확인한다.
- 쓰기 연산의 경우: 버퍼 채널에 빈 공간이 있어서 대기 없이 값을 넣을 수 있는지 확인한다.

만약 나열된 모든 채널이 준비되지 않은 상태라면, select문은 그자리에서 멈춰 선다 (Blocking). 그러다가 하나라도 준비가 완료되는 채널이 생기면, 즉시 해당 연산을 수행하고 그 case 밑에 있는 코드들을 실행한다.

5초 뒤에 채널이 닫히는 신호를 감지하고 탈출하는 전체 코드를 확인해 보자.

```

package main

import (
    "fmt"
    "time"
)

func main() {
    start := time.Now()
    c := make(chan interface{})

    go func() {
        time.Sleep(5 * time.Second)
        close(c) // 5초 대기 후 채널을 닫아 신호를 보냄
    }()
}

```

```

    fmt.Println("Blocking on read...")
    select {
    case <-c: // 채널이 ready(닫힘) 상태가 될 때까지 여기서 대기합니다.
        fmt.Printf("Unblocked %v later.\n", time.Since(start))
    }
}

```

위 코드를 실행하면 다음과 같은 결과가 출력된다.

```

Blocking on read...
Unblocked 5.000170047s later.

```

결과를 보면 select 문에 진입한 후 정확히 약 5초 뒤에 블로킹이 풀린 것을 볼 수 있다. 어떤 사건이 터지기를 기다리는 매우 효율적이고 단순한 방법이다.

(2) 복수 채널이 동시에 준비될 때의 무작위 법칙

여기서 한 가지 흥미로운 질문이 생긴다.

“만약 select 문 안에 있는 여러 개의 채널들이 동시에 읽거나 쓸 수 있는 ready 상태가 되면 어떻게 될까?” 직접 실험을 통해 확인하겠다.

두 개의 채널을 생성하자마자 바로 닫아서 둘 다 언제든지 읽을 수 있는 ready 상태로 만든 뒤, 반복문을 1,001번 돌리며 카운트를 세어보는 전체코드이다.

```

package main

import "fmt"

func main() {
    c1 := make(chan interface{})
    close(c1)
    c2 := make(chan interface{})
    close(c2)

    var c1Count, c2Count int
    for i := 1000; i >= 0; i-- {

```

```

        select {
        case <-c1:
            c1Count++
        case <-c2:
            c2Count++
        }
    }
    fmt.Printf("c1Count: %d\nc2Count: %d\n", c1Count, c2Count)
}

```

위 코드를 실행하면 다음과 같다.

```

c1Count: 505
c2Count: 496

```

결과를 보면 1,001번의 시도 중 거의 정확하게 절반씩 확률을 나누어 가졌다. 단순한 우연이 아니다. Go언어의 런타임은 select 내에서 여러 case가 동시에 충족될 경우, 의도적으로 의사 무작위 균등 선택 알고리즘을 선택한다. 즉, 준비된 모든 case들이 선택받을 확률은 완벽하게 동등하다.

이 설계의 배경에는 컴퓨터 과학의 아주 깊은 통찰이 담겨 있다. Go 런타임은 개발자가 이 select 문을 어떤 비즈니스 의도로 썼는지, 현실 세계의 어떤 문제를 풀기 위해 채널들을 묶어두었는지 전혀 알 방법이 없다.

런타임이 할 수 있는 최선의 선택은 평균적인 상황에서 언제나 프로그램이 잘 굴러가도록 보장하는 것이다. 만약 무작위가 아니라 무조건 위에 적힌 case만 우선시하도록 설계했다면, 아래쪽 채널은 영원히 선택받지 못해 굶어 죽는 기아 상태나 심각한 타이밍 병목 현상이 터졌을 것이다.

공평하게 주사위를 던지는 무작위 방식을 도입함으로써, 모든 Go 프로그램의 select 문은 평균적으로 고른 고성능을 낼 수 있게 되었다.

(3) 타임아웃 구현 패턴

두 번째 질문이다.

“만약 그 어떤 채널도 영원히 ready 상태가 되지 않는다면 프로그램은 어떻게 해야할까?”

모든 채널이 막혀서 아무 일도 못하는 상황인데 프로그램이 평생 얼어붙어 있을 수는 없다. 일정 시간이 지나면 포기하고 빠져나오는 타임아웃(Timeout)처리를 해야 한다. Go언어의 time 패키지는 채널 기반으로 작동하기 때문에 select 문과 완벽하게 찰떡 궁합을 이룬다.

전체코드를 확인해보자.

```
package main

import (
    "fmt"
    "time"
)

func main() {
    var c <-chan int // 초기화되지 않은 nil 채널이므로 이 채널은 평생 읽을 수 없습니다.

    select {
    case <-c:
        // 이 케이스는 평생 실행되지 않고 블로킹됩니다.
    case <-time.After(1 * time.Second):
        // 1초가 지나면 이 채널로 현재 시간이 쏙 들어오며 ready 상태가 됩니다.
        fmt.Println("Timed out.")
    }
}
```

위 코드를 실행하면 다음과 같은 결과가 출력된다.

Timed out.

time.After 함수는 매개변수로 시간 길이(time.Duration)를 받아서,
그 시간이 지나면 현재 시각 데이터를 내부적으로 던져주는 독립된 채널을 반환한다.

이 성질을 select 문에 하나의 case로 엮어두기만 하면, 메인 채널이 막히더라도 지정한 시간이 되는 순간 타임아웃 case가 발동하며 블로킹을 안전하게 뚫고 나갈 수 있게 된다.

(4) 비블로킹 처리를 위한 default 구문

마지막 질문이다.

“채널이 준비되지 않았을 때 대기방에 멈춰 서서 기다리는 대신, 기다리는 동안 댄 일이라도 하고 싶다면 어떻게 될까?”

select 문은 채널들이 전부 막혀있을 때, 즉시 우회해서 실행할 수 있는 default 구문을 지원한다.

default 구문이 들어가면 select 문은 채널이 준비 안되었다고 해서 멈춰 서지 않고, 즉시 default 블록을 실행한 뒤, select 전체를 빠져나온다.

전체코드를 확인해보자.

```
package main

import (
    "fmt"
    "time"
)

func main() {
    start := time.Now()
    var c1, c2 <-chan int // 둘 다 nil 채널이므로 블로킹 상태입니다.

    select {
    case <-c1:
    case <-c2:
    default:
        // 채널들이 준비되지 않았으므로 대기하지 않고 즉시 이 블록을 실행합니다.
        fmt.Printf("In default after %v\n\n", time.Since(start))
    }
}
```

위 코드를 실행하면 다음과 같은 결과가 출력된다.

```
In default after 1.421µs
```

출력 시간을 보시면 진입하자마자 마이크로초단위의 눈깜짝할 사이에 default문이 실행된 것을 볼 수 있다. 즉 대기(Blocking)없이 즉시 탈출한 것이다.

이 default 구문은 실문에서 for-select 루프 패턴과 결합할 때 상상 초월로 강력해진다. 백그라운드 고루틴이 결과를 가져올 때까지 멍하니 기다리는게 아니라, 루프를 계속 돌며 자기 할 일을 열심히 하다가 매 루프마다 “혹시 결과 채널에 뭐 들어왔나?” 슬쩍 체크하는 방식으로 구현할 수 있다.

전체 코드를 보자.

```
package main

import (
    "fmt"
    "time"
)

func main() {
    done := make(chan interface{})

    go func() {
        time.Sleep(5 * time.Second)
        close(done) // 5초 뒤에 하던 일 끝났다고 종료 신호를 보냄
    }()

    workCounter := 0
    loop:
        for {
            select {
            case <-done:
                // 백그라운드 고루틴이 종료 신호를 보내면 전체 반복문을 탈출합니
                다.
                break loop
            default:
                // 신호가 아직 안 왔다면 대기하지 않고 그냥 지나갑니다.
            }
        }
    }
```

```

        // 뇌 시뮬레이션: 기다리는 동안 열심히 나만의 단위 작업을 수행합니다.
        workCounter++
        time.Sleep(1 * time.Second)
    }

    fmt.Printf("Achieved %v cycles of work before signalled to stop.\n", workCounter)
}

```

(5) 영원히 멈추는 특수 케이스

select 문을 쓸 때 아주 독특한 예외 케이스가 하나 있다. 내부에 단 하나의 case도 적지 않고 텅 비워두는 형태이다.

```

package main

import "fmt"

func main() {
    fmt.Println("This program will block forever...")

    // 텅 빈 select 문 사용
    select {}

    // 이 아래 코드는 영원히 도달할 수 없습니다.
}

```

위 코드를 실행하면 다음과 같은 시스템 전면 데드락 결과가 출력되며 프로그램이 강제 종료된다.

```

This program will block forever...
fatal error: all goroutines are asleep - deadlock!

goroutine 1 [select (no cases)]:
main.main()
    /tmp/main.go:9 +0x4f

```

아무 조건도 없기 때문에 메인 고루틴은 이 자리에서 영원히 잠들어 버린다.

메인 함수가 더 이상 깨어날 수 없으므로 Go 런타임이 데드락 에러를 내며 프로세스를 종료시킨다.

(6) 결론

여기까지 고루틴을 묶어주는 핵심 컨트롤 타워인 select 문의 작동 원리와 3가지 핵심 질문에 대한 해답을 정리했다.

select 문을 자유자재로 다룰 수 있게 되면, 여러 개의 하위 시스템들이 보내오는 채널 신호들을 안전하고 효율적으로 한 품에 안아 조율할 수 있게 된다.

이 탄탄한 기본형 벽돌들을 가지고 다음 장부터는 현실의 대규모 아키텍처에서 쓰이는 고급 동시성 디자인 패턴들을 본격적으로 구현해보자.

[03-5] GOMAXPROCS

(1) GOMAXPROCS 이름에 숨겨진 오해

Go언어의 runtime 패키지 안에는 GOMAXPROCS라는 아주 유명한 함수가 존재한다. 저자의 개인적인 의견으로는 이 함수의 이름이 개발자들에게 다소 오해를 불러일으키기 쉽다고 생각한다.

이름만 보면 컴퓨터에 장착된 물리적 하드웨어의 ‘논리 프로세스(코어) 개수’를 설정하는 함수 처럼 보이기 때문이다. 물론 간접적으로는 하드웨어 코어 개수와 밀접한 연관이 있는 것이 맞지만, 실제 이 함수가 통제하는 본질은 ‘작업 대기열(Work Queue)을 올려 놓고 실행할 실제 운영체제 스레드(OS Thread)의 최대 개수’를 제어하는 것이다.

과거 Go 1.5버전 이전 시절에는 이 GOMAXPROCS 값이 기본적으로 언제나 무조건 1로 고정되어 있었다.

즉, 컴퓨터에 코어가 4개든 8개든 Go 프로그램은 단 하나의 코어 위에서만 뱅글뱅글 돌아갔던 것이다.

이 때문에 그 시절 대부분의 Go 프로그램 소스코드 맨 위에는 하드웨어 자원을 남김없이 쓰기 위해 다음과 같은 코드 관례처럼 박혀있다.

```
package main

import (
    "fmt"
    "runtime"
)

func main() {
    // 과거 Go 1.5 이전 버전에서 모든 CPU 코어를 다 쓰기 위해 작성하던 관례 코드
    runtime.GOMAXPROCS(runtime.NumCPU())

    fmt.Printf("Current GOMAXPROCS: %d\n", runtime.GOMAXPROCS(0))
}
```

위 코드를 현재 사용하는 8코어 컴퓨터 환경에서 실행하면 다음과 같은 결과가 출력된다.

```
Current GOMAXPROCS: 8
```

대부분의 개발자는 자기가 만든 프로그램이 실행되는 장비의 모든 코어 파워를 한 방울도 남김없이 빨아먹기를 원한다. 이 피드백을 적극 수용한 Go언어 개발진은 이후 버전부터 컴퓨터가 가진 논리 CPU 개수만큼 자동으로 GOMAXPROCS 값을 알아서 설정해 주도록 엔진을 변경했다. 덕분에 현대의 Go 개발자들은 이 코드를 수동으로 적지 않아도 기본적으로 최고 성능을 낼 수 있게 되었다.

(2) 이 값을 개발자가 직접 조절해야 하는 이유

“알아서 잘 돌아가는데, 그럼 이 값을 우리가 손으로 직접 건드릴 일이 있을까?”

결론부터 말하자면 “평소에는 절대로 건드릴 일이 없으며, 건드리지 않는 것이 낫다”가 정답이다. Go 언어 내부 스케줄링 알고리즘은 상상 이상으로 영리하고 정교하기 때문에, 무턱대고 이 작업 큐와 스레드 개수를 임의로 늘리거나 줄였다가 오히려 시스템 컨디션을 망치고 성능에 독이 되는 경우가 훨씬 많다.

하지만 하드웨어 스펙을 직접 넘나들며 이 레버를 조절할 때 엄청난 유용함을 발휘하는 특수한 상황들이 몇 가지 존재한다.

상황1: 꼭꼭 숨어 있는 동시성 버그(데이터 경쟁) 강제로 끄집어내기

저자가 겪었던 실제 프로젝트 일화가 아주 좋은 예시이다.

당시 개발팀은 간헐적으로 테스트가 깨지는 아주 악랄하고 지저분한 경쟁 조건 버그 때문에 골머리를 앓고 있었다.

해당 테스트 시스템이 돌아가는 인프라 장비는 논리 CPU가 딱 4개밖에 없는 환경이었다. 즉, 물리적으로 완전한 동시 실행이 가능한 고루틴의 개수가 최대 4개로 제한되어 있었던 것이다. 타이밍이 너무 절묘하게 맞아 떨어져서 평소에는 데이터 경쟁 버그가 수면 위로 잘 드러나지 않았던 것이다.

이때 저자가 선택한 신의 한 수는, 실제 하드웨어 코어 개수인 4개를 훨씬 초과하는 수치로, GOMAXPROCS 레버를 강제로 올려버리는 것이었다. 하드웨어가 감당할 수 있는 스레드 작업이 대기열의 한계를 인위적으로 찢어서 고루틴들이 서로 자리를 바꾸는 타이밍(문맥 전환 프로세스)을 극단적으로 흔들어 놓은 것이다.

그 결과, 평소에는 100번에 한 번 터질까 말까 하던 동시성 버그들이 테스트 가동 시 수시로 펄펄 터지기 시작했고, 덕분에 팀원들은 숨어있던 데이터 오염 유발 코드를 빛의 속도로 찾아내어 완벽하게 수리할 수 있었다. 버그 수정 속도를 올리기 위해 하드웨어 엔진 가속기를 억지로 과열시킨 훌륭한 디버깅 테크닉이다.

(3) GOMAXPROCS 레버 조절 시 주의할 점

어떤 개발자들은 다양한 수치 실험을 거치면서 “우리 서비스는 GOMAXPROCS를 특정 숫자로 고정해 두었더니 처리 속도가 훨씬 안정적이고 빨라지던데요?”라며 하드웨어 튜닝의 손맛을 자랑하기도 한다. 하지만 저자는 이러한 행동에 대해 강렬한 어조로 주의를 당부한다.

만약 여러분이 이 레버를 굳이 수동으로 미세 조정해서 극한의 시스템 성능을 쥐어짜 내고 있다면, 다음과 같은 지옥의 유지보수 숙제를 평생 짊어져야 한다.

- 소스코드가 한 줄 바뀌는 모든 커밋 순간마다 성능을 재측정해야 한다.
- 서버 컴퓨터의 하드웨어 부품이 바뀌거나 장비 스펙이 달라질 때마다 다시 세팅해야 한다.
- Go 언어 컴파일러의 버전이 한 단계 올라갈 때마다 최적값을 새로 찾아야 한다.

```
package main

import (
    "fmt"
    "runtime"
    "sync"
    "time"
)

func main() {
    // 인위적으로 CPU 코어보다 4배 많은 스레드 작업 큐를 강제 할당
    numCPU := runtime.NumCPU()
    runtime.GOMAXPROCS(numCPU * 4)

    var wg sync.WaitGroup
    fmt.Printf("Testing with GOMAXPROCS = %d\n", runtime.GOMAXPROCS(0))

    start := time.Now()
    for i := 0; i < 1000; i++ {
        wg.Add(1)
        go func() {
            defer wg.Done()
            // 의미 없는 무거운 연산 수행으로 하드웨어를 한계까지 밀어붙임
            _ = math.Pow(2, 10)
        }()
    }
    wg.Wait()
    fmt.Printf("Elapsed time: %v\n", time.Since(start))
}
```

```

        }0
    }
    wg.Wait()
    fmt.Printf("Total execution time: %v\n", time.Since(start))
}

```

이 코드를 실행하면 하드웨어 사양에 묶인 극단적인 밀착 테스트 결과가 출력된다.

```

Testing with GOMAXPROCS = 32
Total execution time: 1.2045ms

```

이 값을 임의로 튜닝한다는 것은, Go 언어가 자랑하는 아름답고 안전한 ‘추상화 계층’의 보호막을 걷어내고 프로그램 코드를 하드웨어 철판 바닥에 억지로 바짝 밀착시키는 행위와 같다.

당장 눈앞의 벤치마크 점수는 몇 점 올릴 수 있을지 몰라도, 미래에 하드웨어나 Go 언어 인프라가 조금만 바뀌어도 시스템 성능이 요동치고 무너지는 장기적인 안정성의 상실이라는 막대한 대가를 치르게 된다.

(4) 결론

GOMAXPROCS는 동시성 프로그램의 두뇌 개수를 통제하는 아주 매력적이고 강력한 레버이다.

하지만 특별한 디버깅 목적이나 철저하게 통제된 극한의 성능 테스트 환경이 아니라면, Go 개발진의 기술력을 믿고 기본 자동 설정 값에 온전히 시스템을 맡겨두는 것이 소프트웨어 아키텍처 측면에서 가장 안전하고 현명한 선택이다.

가급적 고루틴의 생성과 채널 조율이라는 본연의 소프트웨어 로직 설계에만 집중하시고, 밑바닥 하드웨어의 분배 기술은 Go 런타임에게 전적으로 일임하자.